Introduction to Theoretical Computer Science

CS2490 - Session 11: Application of Minimization and Regular Expressions

Pierre Cagne

Fall 2026

September 11, 2026

A non obvious question

Consider the two following languages over the alphabet $\Sigma = \{a, b\}$:

A non obvious question

Consider the two following languages over the alphabet $\Sigma = \{a, b\}$:

- $L_1 = \{w \in \Sigma^* \mid |w| \geq 1\}$

A non obvious question

Consider the two following languages over the alphabet $\Sigma = \{a, b\}$:

- $L_1 = \{w \in \Sigma^* \mid |w| \geq 1\}$
- $L_2 = \{a^n \mid n \geq 1\} \cup \{a^k b a^{k_1} b a^{k_2} \dots b a^{k_m} \mid k \geq 0, m \geq 1, \text{each } k_i \geq 0\}$

A non obvious question

Consider the two following languages over the alphabet $\Sigma = \{a, b\}$:

- $L_1 = \{w \in \Sigma^* \mid |w| \geq 1\}$
- $L_2 = \{a^n \mid n \geq 1\} \cup \{a^k b a^{k_1} b a^{k_2} \dots b a^{k_m} \mid k \geq 0, m \geq 1, \text{each } k_i \geq 0\}$

Claim: They are the same language!

A non obvious question

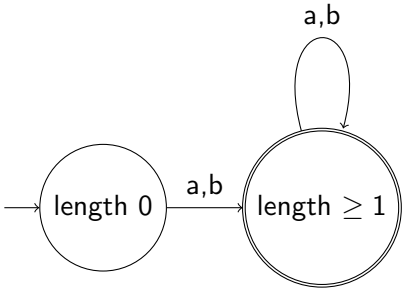
Consider the two following languages over the alphabet $\Sigma = \{a, b\}$:

- $L_1 = \{w \in \Sigma^* \mid |w| \geq 1\}$
- $L_2 = \{a^n \mid n \geq 1\} \cup \{a^k b a^{k_1} b a^{k_2} \dots b a^{k_m} \mid k \geq 0, m \geq 1, \text{each } k_i \geq 0\}$

Claim: They are the same language!

What strategy can you employ to prove this?

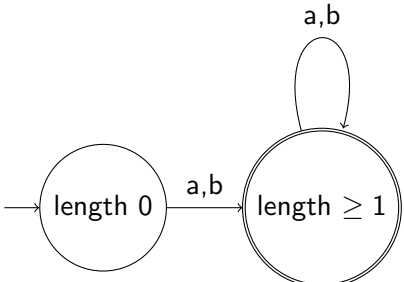
A non obvious question (cont'd)



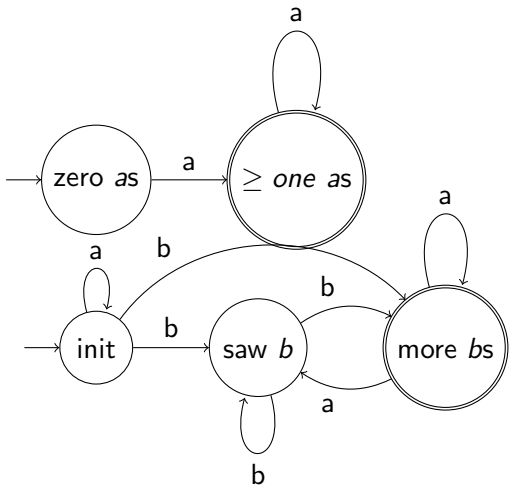
This DFA recognizes L_1.

.

A non obvious question (cont'd)



This DFA recognizes L_1.



This NFA recognizes L_2.

A non obvious question (cont'd)

A non obvious question (cont'd)

- We can minimize the DFA for L_1.

A non obvious question (cont'd)

- We can minimize the DFA for L_1.
- We can convert the NFA for L_2 to a DFA.

A non obvious question (cont'd)

- We can minimize the DFA for L_1.
- We can convert the NFA for L_2 to a DFA.
- We can minimize the resulting DFA for L_2.

A non obvious question (cont'd)

- We can minimize the DFA for L_1.
- We can convert the NFA for L_2 to a DFA.
- We can minimize the resulting DFA for L_2.
- If the two minimal DFAs are identical, then $L_1 = L_2$.

A non obvious question (cont'd)

- We can minimize the DFA for L_1.
- We can convert the NFA for L_2 to a DFA.
- We can minimize the resulting DFA for L_2.
- If the two minimal DFAs are identical, then $L_1 = L_2$.

Let's see it in action!

How to describe regular languages?

$$L_2 = \{a^n \mid n \geq 1\} \cup \{a^k b a^{k_1} b a^{k_2} \ldots b a^{k_m} \mid k \geq 0, m \geq 1, \text{each } k_i \geq 0\}$$

How to describe regular languages?

$$L_2 = \{a^n \mid n \geq 1\} \cup \{a^k b a^{k_1} b a^{k_2} \ldots b a^{k_m} \mid k \geq 0, m \geq 1, \text{each } k_i \geq 0\}$$

This is a bit terse to read!

How to describe regular languages?

$$L_2 = \{a^n \mid n \geq 1\} \cup \{a^k b a^{k_1} b a^{k_2} \ldots b a^{k_m} \mid k \geq 0, m \geq 1, \text{each } k_i \geq 0\}$$

This is a bit terse to read!

A word in L_2 is either:

How to describe regular languages?

$$L_2 = \{a^n \mid n \geq 1\} \cup \{a^k b a^{k_1} b a^{k_2} \ldots b a^{k_m} \mid k \geq 0, m \geq 1, \text{each } k_i \geq 0\}$$

This is a bit terse to read!

A word in L_2 is either:

- a non-empty sequence of as, or

How to describe regular languages?

$$L_2 = \{a^n \mid n \geq 1\} \cup \{a^k b a^{k_1} b a^{k_2} \ldots b a^{k_m} \mid k \geq 0, m \geq 1, \text{each } k_i \geq 0\}$$

This is a bit terse to read!

A word in L_2 is either:

- a non-empty sequence of as, or
- a sequence of as followed by a sequence of blocks, where each block is a b followed by a sequence of as.

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE.

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. No pattern at all!

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. No pattern at all!
- ε is a RE.

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE.

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$ — Either ... or ...

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$ — Either ... or ...
  - $(r_1 r_2)$

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$ — Either ... or ...
  - $(r_1 r_2)$ — Followed by ...

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$ — Either ... or ...
  - $(r_1 r_2)$ — Followed by ...
  - $(r_1{}^*)$

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$ — Either ... or ...
  - $(r_1 r_2)$ — Followed by ...
  - $(r_1{}^*)$ — A sequence of ...

Regular Expressions

Regular expressions (REs) are notations that describe patterns in words.

Let Σ be an alphabet. The REs over Σ are defined *inductively* as follows:

- $\varnothing$ is a RE. — No pattern at all!
- ε is a RE. — Empty pattern
- For each $a \in \Sigma$, a is a RE. — Patterns of length 1
- If r_1 and r_2 are REs, then so are:
  - $(r_1 + r_2)$ — Either ... or ...
  - $(r_1 r_2)$ — Followed by ...
  - $(r_1{}^*)$ — A sequence of ...

We often omit parentheses when there is no ambiguity: * has the highest precedence, then concatenation, then $+$.

Example

What is the RE that describes the pattern of words in L_2?

Example

What is the RE that describes the pattern of words in L_2?

A word in L_2 is either:

Example

What is the RE that describes the pattern of words in L_2?

A word in L_2 is either:

- a sequence

* +

Example

What is the RE that describes the pattern of words in L_2?

A word in L_2 is either:

- a sequence of as

$a^* +$

Example

What is the RE that describes the pattern of words in L_2?

A word in L_2 is either:

- a non-empty sequence of as

$aa^* +$

Example

What is the RE that describes the pattern of words in L_2?

A word in L_2 is either:

- a non-empty sequence of as, or
- a sequence of

$aa^* + \ ^*$

Example

What is the RE that describes the pattern of words in L_2?

A word in L_2 is either:

- a non-empty sequence of as, or
- a sequence of as

$$aa^* + a^*$$

Example

What is the RE that describes the pattern of words in L_2?

A word in L_2 is either:

- a non-empty sequence of as, or
- a sequence of as followed by a sequence of blocks

$$aa^* + a^*(\quad)^*$$

Example

What is the RE that describes the pattern of words in L_2?

A word in L_2 is either:

- a non-empty sequence of as, or
- a sequence of as followed by a sequence of blocks, where each block is a b

$$aa^* + a^*(b\quad)^*$$

Example

What is the RE that describes the pattern of words in L_2?

A word in L_2 is either:

- a non-empty sequence of as, or
- a sequence of as followed by a sequence of blocks, where each block is a b followed by a sequence

$$aa^* + a^*(b\ ^*)^*$$

Example

What is the RE that describes the pattern of words in L_2?

A word in L_2 is either:

- a non-empty sequence of as, or
- a sequence of as followed by a sequence of blocks, where each block is a b followed by a sequence of as.

$$aa^* + a^*(ba^*)^*$$

A little tangent on induction

Inductively indicates a structure with base cases and inductive cases.

A little tangent on induction

Inductively indicates a structure with base cases and inductive cases.

In other words, we have elementary patterns and more complex patterns are obtained by combining previously defined ones.

A little tangent on induction

Inductively indicates a structure with base cases and inductive cases.

In other words, we have elementary patterns and more complex patterns are obtained by combining previously defined ones.

You have already seen inductive definitions before:

A little tangent on induction

Inductively indicates a structure with base cases and inductive cases.

In other words, we have elementary patterns and more complex patterns are obtained by combining previously defined ones.

You have already seen inductive definitions before:

- Binary trees are either empty or a node with two trees.

A little tangent on induction

Inductively indicates a structure with base cases and inductive cases.

In other words, we have elementary patterns and more complex patterns are obtained by combining previously defined ones.

You have already seen inductive definitions before:

- Binary trees are either empty or a node with two trees.
- Linked lists are either empty (`null` in Java) or a node with a `next` linked list.

A little tangent on induction

Inductively indicates a structure with base cases and inductive cases.

In other words, we have elementary patterns and more complex patterns are obtained by combining previously defined ones.

You have already seen inductive definitions before:

- Binary trees are either empty or a node with two trees.
- Linked lists are either empty (`null` in Java) or a node with a `next` linked list.
- Boolean conditions are either true or false, or the negation/conjunction/disjunction of other conditions.

A little tangent on induction

Inductively indicates a structure with base cases and inductive cases.

In other words, we have elementary patterns and more complex patterns are obtained by combining previously defined ones.

You have already seen inductive definitions before:

- Binary trees are either empty or a node with two trees.
- Linked lists are either empty (`null` in Java) or a node with a `next` linked list.
- Boolean conditions are either true or false, or the negation/conjunction/disjunction of other conditions.
- Natural numbers are either zero or the successor of a natural number.

Tying REs to languages

Each RE r describes a language $\mathrm{L}(r)$ over Σ, in the intended way:

Tying REs to languages

Each RE r describes a language $\mathrm{L}(r)$ over Σ, in the intended way:

- $\mathrm{L}(\varnothing) = \emptyset$

Tying REs to languages

Each RE r describes a language $\mathsf{L}(r)$ over Σ, in the intended way:

- $\mathsf{L}(\varnothing) = \emptyset$
- $\mathsf{L}(\varepsilon) = \{\varepsilon\}$

Tying REs to languages

Each RE r describes a language $\mathrm{L}(r)$ over Σ, in the intended way:

- $\mathrm{L}(\varnothing) = \emptyset$
- $\mathrm{L}(\varepsilon) = \{\varepsilon\}$
- For each $a \in \Sigma$, $\mathrm{L}(a) = \{a\}$

Tying REs to languages

Each RE r describes a language $\mathrm{L}(r)$ over Σ, in the intended way:

- $\mathrm{L}(\varnothing) = \emptyset$
- $\mathrm{L}(\varepsilon) = \{\varepsilon\}$
- For each $a \in \Sigma$, $\mathrm{L}(a) = \{a\}$
- If r_1 and r_2 are REs, then:

Tying REs to languages

Each RE r describes a language $\mathrm{L}(r)$ over Σ, in the intended way:

- $\mathrm{L}(\varnothing) = \emptyset$
- $\mathrm{L}(\varepsilon) = \{\varepsilon\}$
- For each $a \in \Sigma$, $\mathrm{L}(a) = \{a\}$
- If r_1 and r_2 are REs, then:
  - $\mathrm{L}(r_1 + r_2) = \mathrm{L}(r_1) \cup \mathrm{L}(r_2)$

Tying REs to languages

Each RE r describes a language $\mathrm{L}(r)$ over Σ, in the intended way:

- $\mathrm{L}(\varnothing) = \emptyset$
- $\mathrm{L}(\varepsilon) = \{\varepsilon\}$
- For each $a \in \Sigma$, $\mathrm{L}(a) = \{a\}$
- If r_1 and r_2 are REs, then:
  - $\mathrm{L}(r_1 + r_2) = \mathrm{L}(r_1) \cup \mathrm{L}(r_2)$
  - $\mathrm{L}(r_1 r_2) = \mathrm{L}(r_1) \cdot \mathrm{L}(r_2)$

Tying REs to languages

Each RE r describes a language $\mathrm{L}(r)$ over Σ, in the intended way:

- $\mathrm{L}(\varnothing) = \emptyset$
- $\mathrm{L}(\varepsilon) = \{\varepsilon\}$
- For each $a \in \Sigma$, $\mathrm{L}(a) = \{a\}$
- If r_1 and r_2 are REs, then:
  - $\mathrm{L}(r_1 + r_2) = \mathrm{L}(r_1) \cup \mathrm{L}(r_2)$
  - $\mathrm{L}(r_1 r_2) = \mathrm{L}(r_1) \cdot \mathrm{L}(r_2)$
  - $\mathrm{L}(r_1^*) = \mathrm{L}(r_1)^*$

Tying REs to languages

Each RE r describes a language $\mathrm{L}(r)$ over Σ, in the intended way:

- $\mathrm{L}(\varnothing) = \emptyset$
- $\mathrm{L}(\varepsilon) = \{\varepsilon\}$
- For each $a \in \Sigma$, $\mathrm{L}(a) = \{a\}$
- If r_1 and r_2 are REs, then:
  - $\mathrm{L}(r_1 + r_2) = \mathrm{L}(r_1) \cup \mathrm{L}(r_2)$
  - $\mathrm{L}(r_1 r_2) = \mathrm{L}(r_1) \cdot \mathrm{L}(r_2)$
  - $\mathrm{L}(r_1^*) = \mathrm{L}(r_1)^*$

We sometimes say that r denotes the language $\mathrm{L}(r)$.

Examples

Examples

- What is the language denoted by $(0 + 1)^*$?

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1. Or L_2.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1. Or L_2.
- What is the language denoted by $a(a + b)^* + b(a + b)^*$?

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ *containing the substring bab.*
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. *I.e.,* L_1. *Or* L_2.
- What is the language denoted by $a(a + b)^* + b(a + b)^*$?
  All non-empty words over $\{a, b\}$.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1. Or L_2.
- What is the language denoted by $a(a + b)^* + b(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1/L_2 again.

Examples

- What is the language denoted by $(0 + 1)^*$?
  All binary words.
- What is the language denoted by $1(0 + 1)^*1$?
  All binary representations of odd numbers but 1.
- What is the language denoted by $(a + b)^*bab(a + b)^*$?
  All words over $\{a, b\}$ containing the substring bab.
- What is the language denoted by $(a + b)(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1. Or L_2.
- What is the language denoted by $a(a + b)^* + b(a + b)^*$?
  All non-empty words over $\{a, b\}$. I.e., L_1/L_2 again.

As for DFA/NFA, multiple REs can denote the same language.

Relation to DFA/NFA

Regular expressions are obviously related to regular languages.

Relation to DFA/NFA

Regular expressions are obviously related to regular languages.

Theorem

Let Σ be an alphabet. A language over Σ is regular if and only if there exists a RE that denotes it.

Relation to DFA/NFA

Regular expressions are obviously related to regular languages.

Theorem

Let Σ be an alphabet. A language over Σ is regular if and only if there exists a RE that denotes it.

Proof.

($\Leftarrow$) From a RE r, construct a NFA $\mathcal{A}_r$ that recognizes $\mathrm{L}(r)$.

Relation to DFA/NFA

Regular expressions are obviously related to regular languages.

Theorem

Let Σ be an alphabet. A language over Σ is regular if and only if there exists a RE that denotes it.

Proof.

($\Leftarrow$) From a RE r, construct a NFA $\mathcal{A}_r$ that recognizes $\mathrm{L}(r)$.

($\Rightarrow$) From a DFA/NFA $\mathcal{A}$, construct a RE $r_{\mathcal{A}}$ that denotes $\mathrm{L}(\mathcal{A})$.



Relation to DFA/NFA

Regular expressions are obviously related to regular languages.

Theorem

Let Σ be an alphabet. A language over Σ is regular if and only if there exists a RE that denotes it.

Proof.

($\Leftarrow$) From a RE r, construct a NFA $\mathcal{A}_r$ that recognizes $\mathrm{L}(r)$. Next time.

($\Rightarrow$) From a DFA/NFA $\mathcal{A}$, construct a RE $r_{\mathcal{A}}$ that denotes $\mathrm{L}(\mathcal{A})$.

□

Relation to DFA/NFA

Regular expressions are obviously related to regular languages.

Theorem

Let Σ be an alphabet. A language over Σ is regular if and only if there exists a RE that denotes it.

Proof.

($\Leftarrow$) From a RE r, construct a NFA $\mathcal{A}_r$ that recognizes $\mathrm{L}(r)$. Next time.

($\Rightarrow$) From a DFA/NFA $\mathcal{A}$, construct a RE $r_{\mathcal{A}}$ that denotes $\mathrm{L}(\mathcal{A})$. On Fri.

□